COLLEGE PROJECT PRESENTATION · CYBERSECURITY / CRYPTOGRAPHY

RSA-SHIELD

An Intelligent RSA Key Security Auditor for Legacy Critical Infrastructure

Software auditing platform using established cryptography (RSA + Lenstra ECM) — not a new cryptosystem.

Team Aegis

Nishant Agarwal	S24CSEU1512
Adarsh Agrahari	S24CSEU1187
Shubh Jadiya	S24CSEU0530

Mentor: **Dr. Sachin Choudhary**
Institution: **Bennett University**

$$n = p \times q$$

RSA security depends on secrecy of the primes behind the modulus.

Audit outcome tiers

Factor Found — hard evidence
Suspicious — heuristic warning
Inconclusive — no factor in budget
No weakness by configured checks

Evidence is never confused with heuristics.

Project objective

Build a software-only cybersecurity platform that accepts RSA public keys/moduli, analyzes cryptographic properties, attempts mathematical factorization using Lenstra's Elliptic Curve Method (ECM) where appropriate, identifies suspicious or weak RSA configurations, and generates an understandable, evidence-tiered security assessment for authorized infrastructure owners.

Why it matters

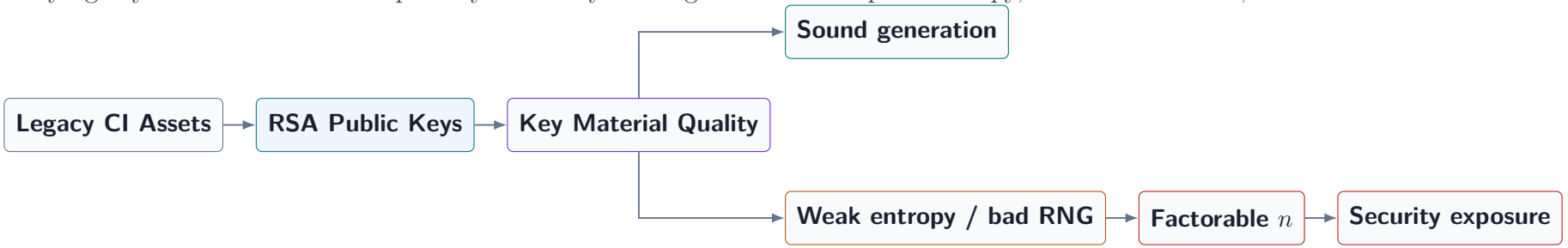
Legacy SCADA/substation systems may still encrypt successfully while hiding weak key material. RSA-SHIELD helps owners move from simple key inventories to **mathematical assurance**—without claiming to invent RSA or ECM.

Audience

Faculty, cybersecurity evaluators, technical judges, and software engineering reviewers.

RSA Keys Can Fail Before the Encryption Does

Critical infrastructure (SCADA, substations, industrial gateways) often retains **legacy cryptographic assets**. RSA may still "work" operationally while the underlying key material is weak—especially when keys were generated with poor entropy, outdated libraries, or constrained embedded RNGs.



Silent mathematical failure

- RSA rests on hardness of factoring $n = p \times q$
- Small/structured factors can make n breakable via ECM
- Device may still encrypt/decrypt normally
- Weakness is mathematical, not a visible outage

Legacy inventory pain

- Years of certs, device keys, VPN/gateway keys
- Generation history often unknown
- Manual review cannot scale
- Bit-length alone misses weak primes inside "large" n

What auditors need

- Structural + mathematical checks
- Concrete evidence when a factor appears
- Clear tiered labels for findings
- Exportable reports for remediation

From Key Inventory to Mathematical Assurance

Typical existing practice

1. Collect keys into inventories / spreadsheets
2. Check bit length (1024 / 2048 / 4096)
3. Check public exponent / basic config flags
4. Mark "OK" if length looks acceptable

Gap: length $\neq$ mathematical strength. Weak primes can hide inside a large-looking modulus.

RSA-SHIELD approach

1. Parse & validate RSA public key / modulus
2. Structural and parameter analysis
3. Budgeted Lenstra ECM factorization attempts
4. Evidence-tiered risk class + JSON/PDF report

Value: established math becomes a practical audit workflow with exportable evidence.

End-to-end audit pipeline



Positioning (Team Aegis): We did **not** invent RSA or ECM. RSA-SHIELD is a security auditing platform that operationalizes known algorithms for authorized assessment at **Bennett University** under **Dr. Sachin Choudhary**.

Tiers: Factor Found (math) · Suspicious (heuristic) · Inconclusive (no factor in budget) · No weakness by configured checks.

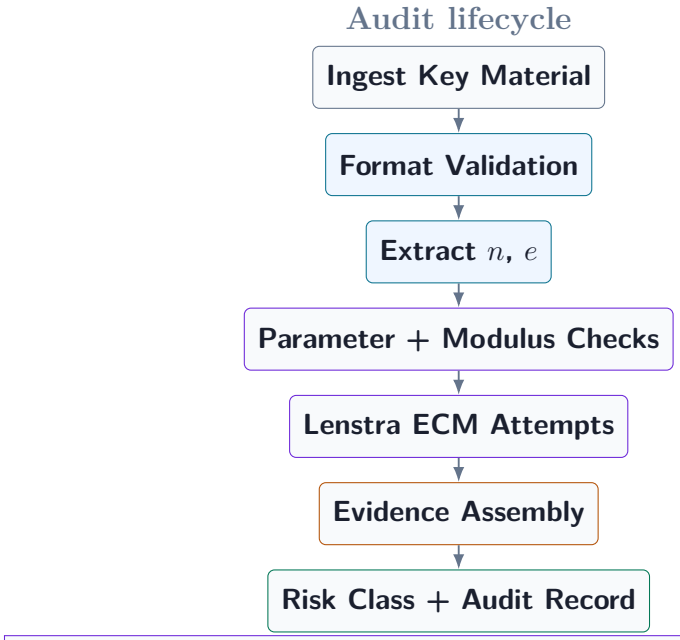
Inside the Audit Engine — How RSA-SHIELD Works

$$n = p \times q$$

Recovering a non-trivial factor of n compromises the RSA modulus and enables private-key recovery.

Detailed processing steps

1. **Ingest** PEM/DER public key or raw modulus integer
2. **Validate** encoding, structure, required fields
3. **Extract** public modulus n and exponent e
4. **Preliminary checks** bit-length, unusual e, trivial divisibility
5. **ECM campaign** configurable curves, bounds, time budget
6. **Factor handling** store non-trivial factors if found
7. **Exposure mapping** evidence → severity class
8. **Persist & report** history + JSON/PDF export



Orchestrator coordinates **Parser**, **Key Analyzer**, **ECM Engine**, **Risk Engine**, and **Report Generator** as separate modules with timeouts and audit logging.

Why Lenstra's Elliptic Curve Method (ECM)?

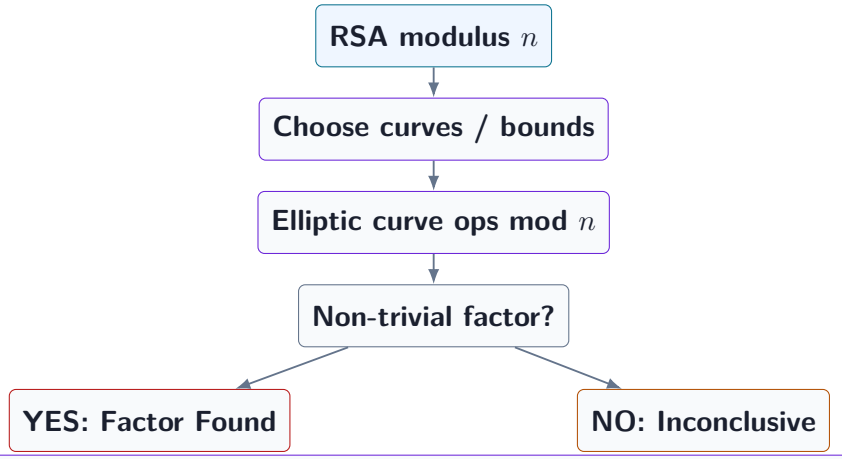
Lenstra ECM is a classic integer-factorization algorithm. It is especially effective when a large composite has a **relatively small** prime factor—a failure mode that can appear in poorly generated RSA moduli used by legacy devices.

Auditor's conceptual view

- Sample random elliptic curves modulo n
- Curve arithmetic can fail when a factor of n divides a bad denominator
- A non-trivial gcd with n reveals a factor = strong evidence
- No factor within budget ⇒ **inconclusive**, not "proven secure"

Honest limit: ECM does not instantly factor properly generated modern RSA (e.g. strong 2048-bit primes). RSA-SHIELD uses ECM as a **weakness probe**, not a universal RSA break.

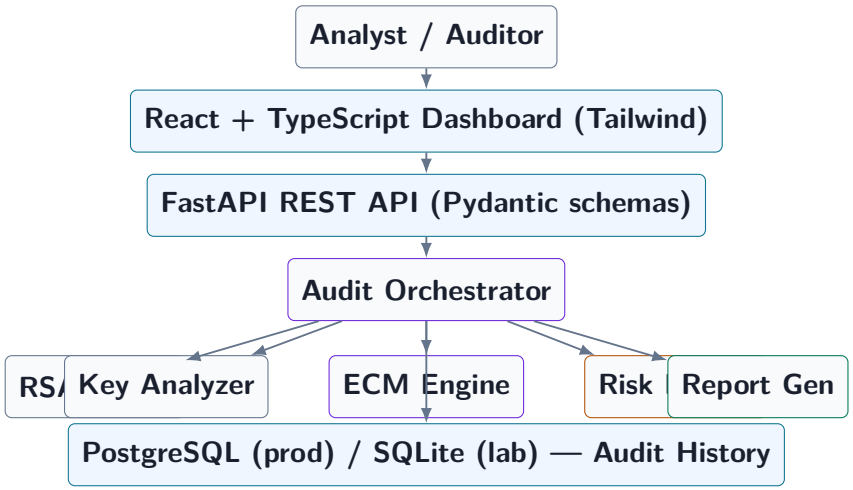
ECM audit workflow



Why this fits legacy CI

Old embedded RNGs and rushed key generation can leave small factors. Finding one factor proves the modulus unsafe and supports immediate key rotation—with evidence, not guesswork.

RSA-SHIELD System Architecture



Frontend

- Key upload & batch queue
- Status / severity badges
- Pipeline visualizations
- History + report download

Backend

- Validated API contracts
- ECM job timeouts/budgets
- Evidence artifact storage
- JSON + PDF assembly

Deployment

- Dockerized services
- SQLite demos; Postgres labs
- GitHub CI with Pytest
- Environment-based config

Layered Analysis Engine and Authorized Audit Scenario

Six analysis layers

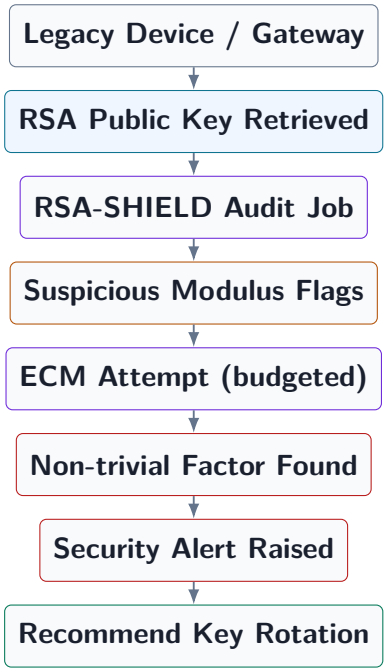
1. **Format & parameter validation** — PEM/DER sanity
2. **RSA parameter analysis** — e, size, anomalies
3. **Modulus characteristics** — trivial factor screens
4. **ECM factorization attempt** — budgeted math probe
5. **Evidence-based risk class** — proof vs heuristic
6. **Audit report** — human + machine readable

Principle: mathematical evidence (factor found) is never mixed with soft warnings without explicit labeling.

Mock dashboard (illustrative)

Key ID	`LEGACY-SCADA-0042`
Modulus	1024-bit
Exponent	$e = 65537$
ECM status	Factor discovered
Severity	CRITICAL
Action	Recommend key rotation

Authorized scenario (owner consent / lab keys)



Factor-discovery claims apply only when demonstrated on authorized or intentionally weak test keys.

Technology Stack, Product Features, Evaluation Plan

Technology stack

UI	React, TS, Tailwind, Vite React Flow / Recharts
API	Python, FastAPI, Pydantic
Engine	RSA math, modular arith. Lenstra ECM modules
Data	PostgreSQL / SQLite
DevOps	Docker, Git, GitHub
Test	Pytest unit+integration

Product features

- RSA key / modulus upload
- Batch key auditing
- Modulus & parameter analysis
- Configurable ECM attempts
- Audit history & evidence store
- Risk classification badges
- JSON + PDF reports
- API for workflows
- Audit simulation mode
- Interactive visualizations

Evaluation metrics

- Parse success rate
- Audit latency
- ECM wall-clock time
- Detection on weak lab keys
- Heuristic FP rate
- Batch throughput
- CPU/memory use

Prototype Benchmark

To Be Measured

— no fabricated numbers —

Evaluation discipline: detection rates are reported only for **intentionally weak test keys** under authorized lab conditions at Bennett University. ECM failure on a strong key is recorded as **inconclusive**, never as a security proof.

What RSA-SHIELD Does — and Does Not — Claim

What Team Aegis claims

- Accepts/parses RSA public keys and moduli
- Performs structural and parameter analysis
- Runs budgeted Lenstra ECM as a weakness probe
- Treats a discovered factor as **hard evidence**
- Separates evidence from heuristic warnings
- Produces understandable JSON/PDF assessments
- Supports authorized inventory audit workflows

Goal: help owners find weak legacy keys before adversaries do.

What we explicitly do not claim

- ECM is not a magic attack on modern strong RSA
- No factor found $\neq$ proof the key is secure
- Tool is an auditing aid, not a formal security proof
- Production needs specialist cryptographic review
- Only **authorized** keys/infrastructure may be audited
- We did not invent RSA, ECM, or a new cipher

Credibility comes from precise, responsible claims.

This strengthens the project for evaluators: Team Aegis understands both the **power** and the **limits** of factorization-based auditing. Responsible framing is part of the engineering contribution under mentorship of **Dr. Sachin Choudhary** at **Bennett University**.

From RSA Auditor to Cryptographic Asset Intelligence

Near-term RSA-SHIELD focuses on single-key and batch modulus auditing. The roadmap expands toward continuous cryptographic asset intelligence for enterprises and critical-infrastructure operators.

Large-scale key inventory | **Certificate scanning** | **Weak-key pattern detection**

Historical key monitoring | **Hardware key metadata** | **PQC migration readiness**

Enterprise dashboards | **CI/CD crypto scanning** | **Compliance report packs**

Inventory scale

Fleet-wide discovery of RSA assets across devices, certificates, and services.

Continuous assurance

Track key health over time; alert when new weak patterns appear.

Enterprise readiness

Dashboards, CI gates, and compliance packs for operators.

Know Your Keys. Prove Their Strength.

Team Aegis

Nishant Agarwal (S24CSEU1512) Adarsh Agrahari (S24CSEU1187) Shubh Jadiya (S24CSEU0530)

Mentor: **Dr. Sachin Choudhary** Institution: **Bennett University**

GitHub: https://github.com/team-aegis/rsa-shield